AIORI Hackathon 3 • PS-024 • Cybersecurity

TokenForge

Privacy Pass Token Issuance and Anti-Replay Gateway

Team TokenForge • A3-PS024-TC119
Krish Sonpal, Mihir Goswami
Pandit Deendayal Energy University
Faculty Guide: Dr. Kaushal Shah

Blind issuance

RFC 9578 Type 2, Blind RSA

Public verification

Gateway checks tokens with a public key

Atomic single spend

Each token works only once

Performance evaluation plan

Latency, throughput, memory

Problem Description

Many users share one IP address, so blocking an IP also blocks innocent users.

Corporate NAT

Many employees go out through one public IP. Block it for one bad request and everyone is blocked.

Commercial VPNs

Many subscribers share one VPN exit IP. One abuser can get the IP banned for all.

Carrier-grade NAT

Internet providers put many customers behind one public IP address.

iCloud Private Relay

The relay hides the user's IP. The website sees only a relay IP shared by many users.

Mobile / 5G

Many phones share an IP and a phone's IP can change, so IP history is unreliable.

Why common alternatives are not enough [12]

- **Device fingerprinting:** tracks a user across sites
- **CAPTCHA:** hard for some users and for API clients [12]
- **Cookies, sessions:** link a user's activity together
- **Client certificates:** need enrolment and identify the user

What a correct solution needs

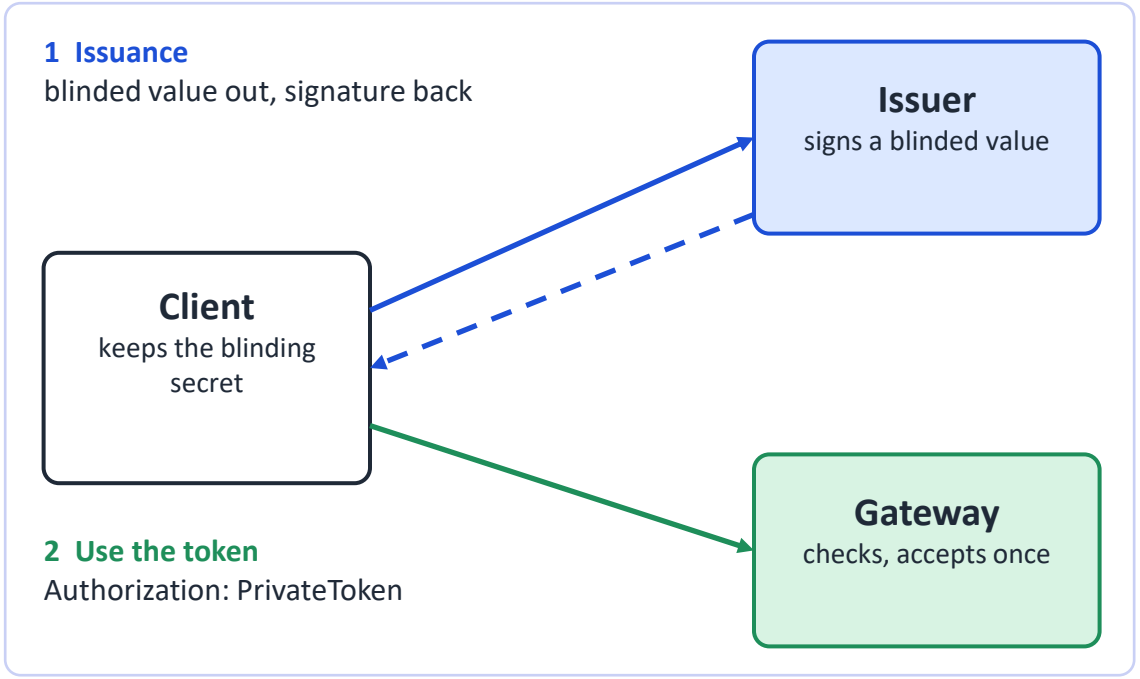
- Check a client is allowed, without learning who it is
- Accept each token only once
- Reject forged tokens and tokens for another service
- Keep token issuing unlinkable from token use
- Add little delay and store little metadata

In short: *NAT lets many devices share one public IP. Our fix is a one-time token that proves "allowed" without revealing who you are.*

Background

How a Privacy Pass token works

- **Step 1, get a token [7]:** the client blinds a secret value, the issuer signs it without seeing it, and the client unblinds it into a valid token.
- **Step 2, use the token [3]:** the client sends it to the gateway in an HTTP Authorization header.
- **Step 3, gateway checks:** valid signature, right service and time (context), and not used before.
- **Why it is private:** the issuer never sees the final token, so it cannot link it to the user. Timing and logs can still leak links.
- **Why Type 2 [4][5]:** the gateway verifies with a public key, so the secret signing key stays with the issuer.



RFC 9576 Architecture [2]

Defines the roles: client, issuer, origin. In our lab, a controller hands out one-use vouchers as permission to get a token.

RFC 9577 HTTP scheme [3]

How tokens travel in HTTP: the server sends a challenge (WWW-Authenticate), the client replies with the token (Authorization: PrivateToken).

RFC 9578 Issuance [4]

Two token types. Type 1 (VOPRF): the verifier needs the secret key. Type 2 (Blind RSA): the verifier needs only a public key. We use Type 2.

Literature & RFC Review

Source	Takeaway for TokenForge
Chaum, blind signatures [7]	Foundation: unlinkable signed credentials
Davidson et al., PoPETs 2018 [8]	Privacy Pass design, deployment and performance evidence
Henry & Goldberg 2011 [9]	Anonymous blacklisting: issuance eligibility is separate from redemption
RFC 9576 / 9577 / 9578 [2][3][4]	Architecture, HTTP binding, Type 1 and Type 2 issuance
RFC 9474 / 9497 [5][6]	Blind RSA (public verify) and OPRF/VOPRF primitives
pat-go, privacypass-ts [11]	Ready-made token libraries; none is a complete, tested gateway for our setup
Apple iCloud Private Relay [10]	Production use with disclosed replay and logging policies

Already exists: anonymous token issuance, IETF standards and open-source libraries.
Our proposed contribution: a reproducible TokenForge deployment, tested against reuse, forged tokens, wrong service and shared-IP access.

Gaps our work addresses

GAP-1: One gateway with a clear key boundary

GAP-2: A reproducible shared-IP test

GAP-3: Real latency and throughput numbers

GAP-4: A field-by-field metadata audit

GAP-5: Tests for forged, reused and wrong-service tokens

GAP-6: Replay-cache lifetime matches token validity

GAP-7: Strict RFC conformance

GAP-8: Clear limits on what we claim

Proposed Solution

The client gets a one-use token from the issuer, then spends it once at the gateway

Issuer Service

RFC 9578 Type 2, RSA-2048. Signs blinded requests; the secret key never leaves it. A one-use lab voucher allows one signing.

Addresses GAP-1, GAP-7, GAP-8

Client Harness

Five isolated test clients. Token = type, nonce, challenge_digest, token_key_id, authenticator.

Addresses GAP-2, GAP-5

Verification Gateway

Checks the signature with the public key, checks service and time, records first use in Redis, then forwards once.

Addresses GAP-1, GAP-5, GAP-6

Replay Store (Redis)

Atomic Lua script: SET … NX PXAT. Entries expire at epoch end + 30 s. If state is uncertain, the run stops.

Addresses GAP-5, GAP-6

Shared Egress Proxy

Tinyproxy tunnels all clients to the issuer and gateway, so they share one source IP. It cannot read the traffic.

Addresses GAP-2

Origin + Lab Controller

Origin: private backend that counts arrivals. Controller: sets up keys and vouchers, resets the run, collects results.

Addresses GAP-1, GAP-3, GAP-8

Why Type 2: Type 1 verification needs the issuer's secret key. Type 2 needs only the public key, so the signing key stays isolated [4][5].
Stack: TypeScript, Node.js 24, @cloudflare/privacypass-ts 0.9.0 [11], Redis 7.4 + ioredis 6.0.0, Tinyproxy, Docker Compose.

Gateway Admission and Decisive Tests

1

Strict parse

base64url, exactly 354 B, no trailing bytes

400 token-malformed

2

Key match

token's key ID matches the trusted public key

401 token-key-unknown

3

Verify signature

RSA-PSS check of the token with the public key

401 token-invalid

4

Context check

digest of the current challenge must match (right service and time)

401 token-context-mismatch

5

Atomic claim

Redis Lua: check the time window and record first use in one step [13]

401 token-already-redeemed or token-expired; 503 if state is unsure

6

Forward once

only after a confirmed claim; credentials stripped; no retry

reaches the origin

Acceptance and retention policy

- 10-minute windows (epochs); only the current one is accepted
- Spent entries stay until window end + 30 s, so deleting them never allows reuse
- If Redis state or the clock is uncertain, stop and restart with a fresh key

Decisive test	Expected result
Valid token	200; one origin arrival
Tampered authenticator	401 token-invalid; 0 arrivals
Genuine token for another origin	401 token-context-mismatch; 0 arrivals
Replay: sequential and concurrent, 2 workers	One accepted, repeats rejected; at most 1 origin arrival
Shared proxy: 5 clients × 2 tokens	10 accepted; replay rejected; one source IP

Optimization Proposed

Area	Naive version (for comparison)	Proposed B1 (our design)	Expected effect (to be measured)
Gateway admission	One process, one request at a time	2 worker processes, 2 active requests each; queue of 16 per worker, 100 ms wait limit	Up to 4 requests in parallel; speed-up to be measured
Issuer key at gateway	Fetch and import the issuer key on every request	Import the pinned public key once per worker and run	No key fetch per request
Replay state	Unbounded set; separate GET then SET (race condition)	One atomic Redis script; entries expire at epoch end + 30 s; 256 MiB cap	Bounded memory; no race
Connections	New connection for every operation	Reused connections: 2 Redis and 2 origin sockets per worker	Less setup time
Overload	Unbounded waiting requests	Queue limit and deadlines; busy reply (503) before any token is spent	Bounded memory; nothing spent
Issuance	One request per token	Unchanged: one token per request (RFC 9578 has no batch format)	No gain claimed

Not included: batch issuance, batch DLEQ proofs, Bloom filters (false positives would reject honest users), token caches, ARC. **No speed-up numbers are claimed yet. We will measure P50/P95/P99 at 10, 50 and 100 concurrent clients against a safe one-worker baseline (B0) that uses the same checks.**

Solution Architecture & Design

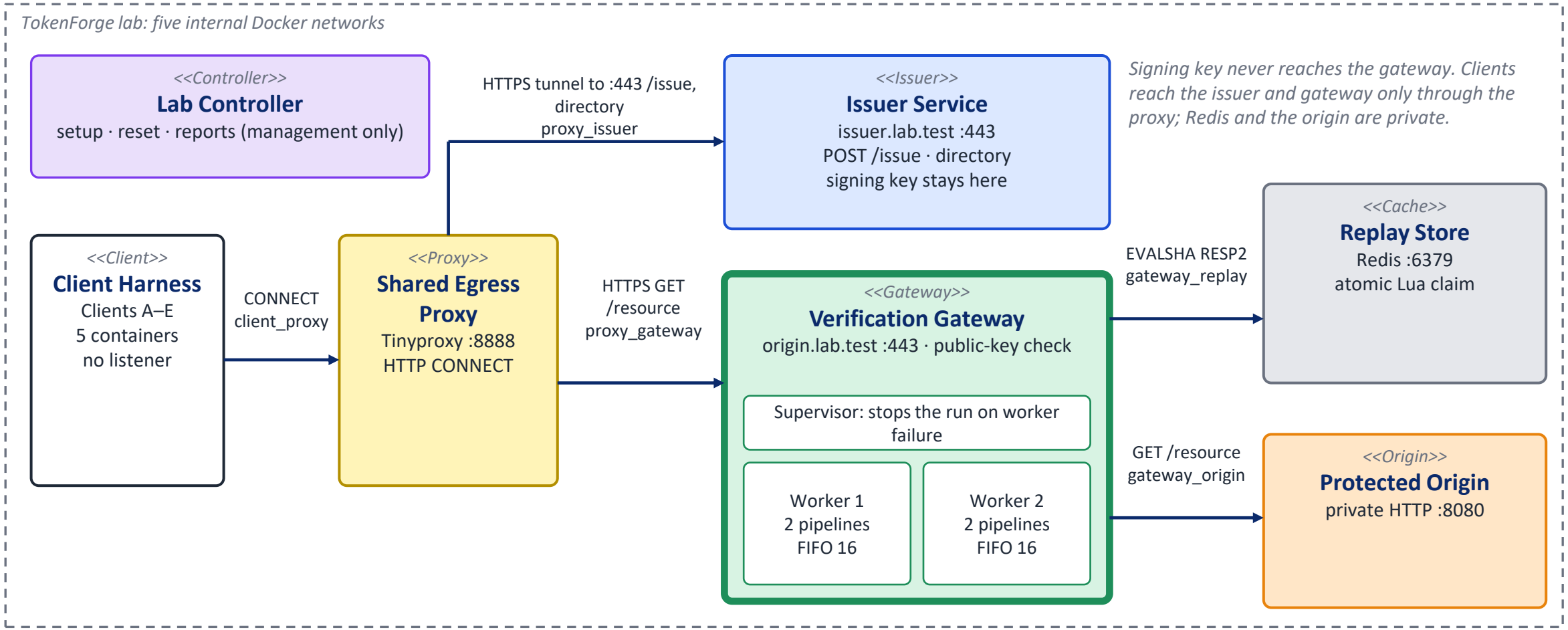


Figure: System architecture overview. Bold border: the gateway. The controller only handles setup and management.

Data Flow Diagram: Issuance

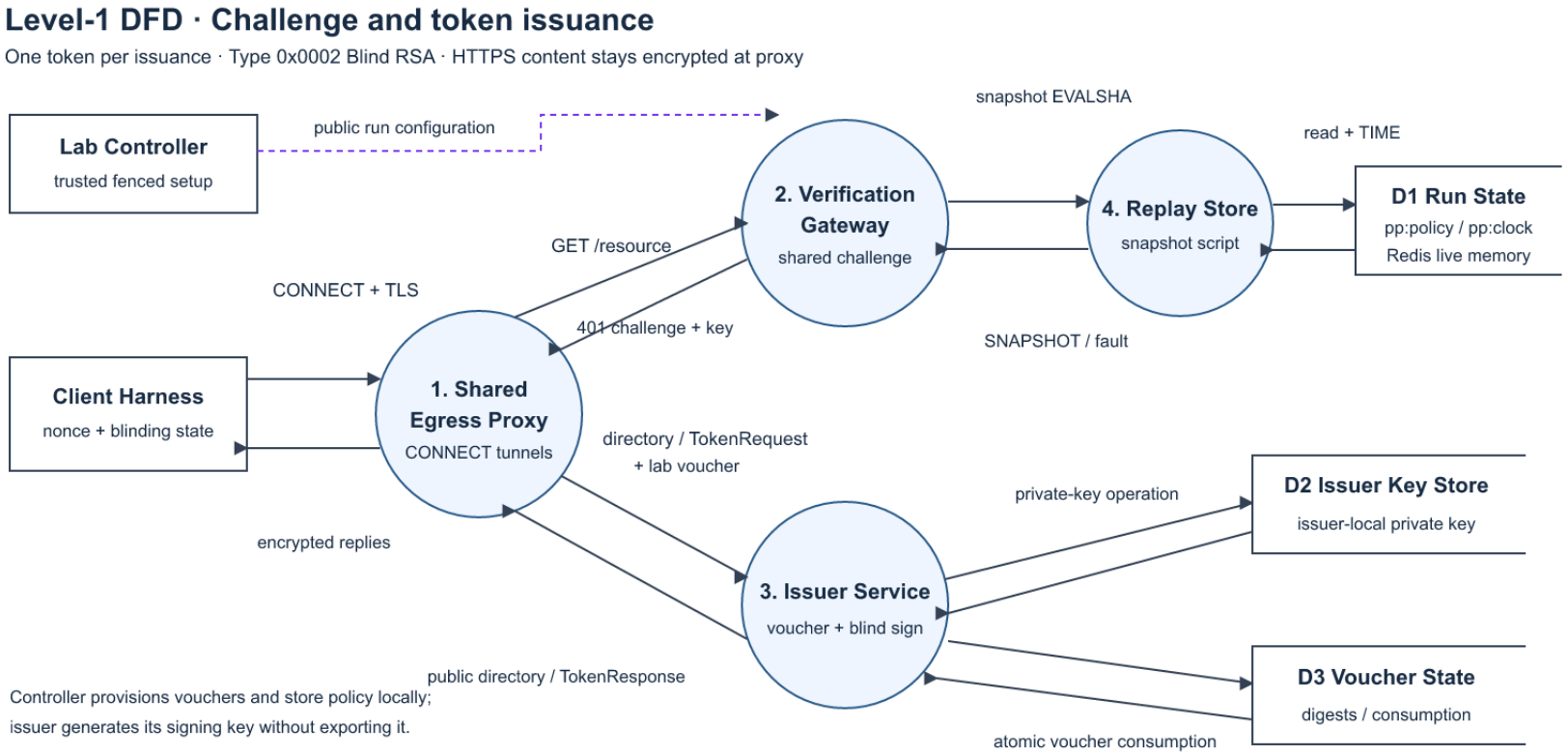


Figure: Level-1 DFD, challenge and token issuance. Rectangles: external entities · circles: processes · open rectangles: data stores · dashed: management setup.

How to read it

- Gateway sends a challenge; the client blinds its token input
- Client sends one request with a one-use voucher (259 B)
- Issuer checks the voucher and signs blindly (256 B reply)
- Client unblinds and checks the signature to get the token
- Signing key stays in the Issuer Key Store; issuing creates no spent record

Data Flow Diagram: Redemption

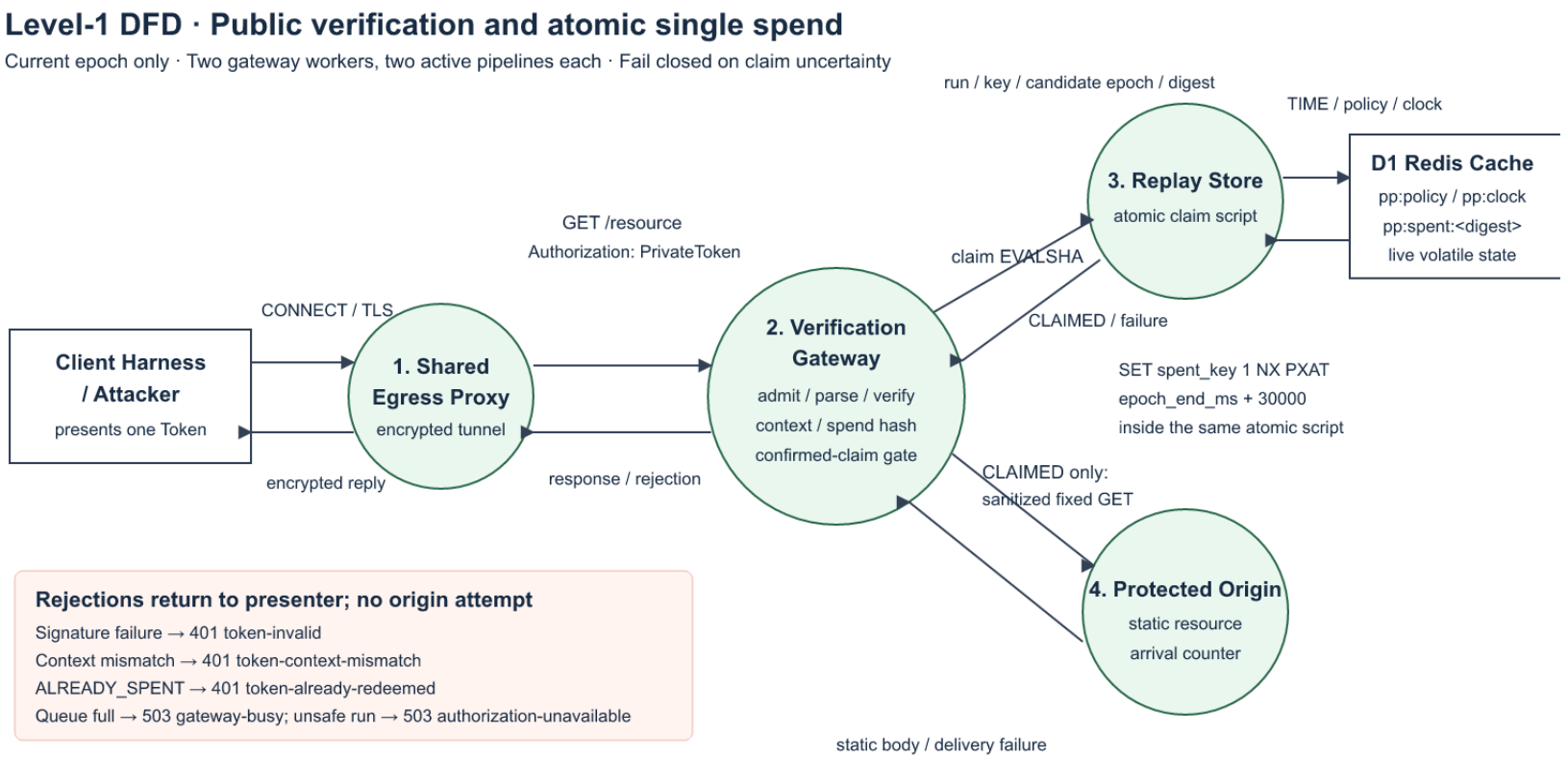


Figure: Level-1 DFD, public verification and atomic single spend. Redis arrows are internal data access, not a separate network GET then SET.

How to read it

- Gateway parses, verifies the signature, then checks service and time
- Redis claims the token atomically; only a confirmed claim forwards one request
- Forged: token-invalid. Wrong service: token-context-mismatch. Reuse: token-already-redeemed
- Same source IP but different tokens are handled independently
- If a claim is uncertain, the system stops (fails closed)

Threat Model & Unlinkability

Threat	Control and outcome
Forgery, wrong origin	Signature checked with the public key; service and time checked; rejected before any token is spent
Replay across workers	One atomic Redis claim per token (hash of the full token input): at most one forwarding attempt
State loss, stale key	System stops; old workers are shut down; restart with a fresh key and run
Issuer and verifier collusion	Blind signing stops them matching a signing to a token by crypto alone; timing and shared logs can still link
Stolen unused token	Cannot be prevented: the first holder to spend it wins; it expires with its time window
Timing, saturation	Timing is observable; no full-anonymity or DDoS claim; queues limit load (503)

What privacy we claim

Blind issuance stops the issuer from matching a signing request to a later token using the crypto transcripts alone, under the scheme's assumptions.

Timing, enrolment records and connection details can still link activity.

Not protected: source IP, timing correlation, enrolment metadata, volume. **Assumed:** trusted gateway, Redis, controller and host; correct client, library and randomness; intact replay state; non-regressing clock.

Metadata-Retention Analysis

Keep only what replay protection needs; avoid records that can link users.

Component	Minimum retained state	Lifetime
Issuer Service	Signing key; unused voucher digests	Active run; voucher removed on one-attempt use
Client Harness	Own pending state, vouchers, usable tokens	Until use, context end or run closure
Verification Gateway	Trusted public key; one shared current context; aggregate metrics	Key until the run ends; context replaced each epoch
Replay Store	Spent-token hash; policy and clock guards	Hash: until epoch end + 30 s (about 30–630 s after use). Guards: the run
Origin, proxy, controller	Arrival count; live tunnels; summary report	Run duration; report kept as project record

Expiring spent entries before their time window ends would allow reuse, so retention cannot be cut shorter. Timing and known-token correlation remain possible. These are proposed requirements we will audit. [GAP-3, GAP-4, GAP-6]

Rules

- No routine token, voucher, blinded-message, IP or per-request timestamp logs
- No Redis persistence, replicas or exported spent keys
- Fresh key per run; old contexts stay rejected
- Redis expiry bounds logical state, not forensic memory

Deployment Timeline

Phase	Needs first	Build and optimization	Milestone
0 Setup	None	Private repo (add aiori-hackathon), Compose skeleton with 5 networks, pin privacypass-ts 0.9.0, hello-world token	First token issued and verified in a test
1 Issuer	Setup	Key generation, directory, POST /issue, voucher consumption, strict length checks	RFC test vectors pass
2 Client	Issuer (use a gateway stub)	Challenge fetch, fresh Client per token, finalize, in-memory tokens, proxy transport	End-to-end issuance through the proxy
3 Gateway + replay	Setup	Parser, public verify, digest check, Lua claim, forward once; B1: key reuse, pools, 2 workers, queue	All rejection paths and concurrent replay tests pass
4 Shared proxy	Client, Gateway	Tinyproxy, network isolation, 5 clients × 2 tokens plus replay, origin-bypass tests	Shared-proxy test passes; one source IP seen
5 Measure	Shared proxy	B0 vs B1 benchmarks (P50/P95/P99 at 10/50/100), memory per entry, retention audit	All measurements collected
6 Submit	All phases	PDF to the template, threat model, pseudocode demo, repo transferred to aiori-hackathon	All deliverables complete

Risks: library or RFC mismatch → independent vectors, no HMAC substitute · Redis fault → fail closed, fenced reset · unavailable member → named critical-path owner for the Gateway. Dates are fixed once the deadline is confirmed.

Bibliography

[1] IETF, RFC 6269, Issues with IP Address Sharing, 2011.

[2] IETF, RFC 9576, The Privacy Pass Architecture, June 2024.

[3] IETF, RFC 9577, The Privacy Pass HTTP Authentication Scheme, June 2024.

[4] IETF, RFC 9578, Privacy Pass Issuance Protocols, June 2024.

[5] IRTF/CFRG, RFC 9474, RSA Blind Signatures, October 2023.

[6] IRTF/CFRG, RFC 9497, Oblivious Pseudorandom Functions (OPRFs) Using Prime-Order Groups, December 2023.

[7] D. Chaum, Blind Signatures for Untraceable Payments, CRYPTO '82, 1983.

[8] A. Davidson, I. Goldberg, N. Sullivan, G. Tankersley, F. Valsorda, Privacy Pass: Bypassing Internet Challenges Anonymously, PoPETs 2018(3):164–180.

[9] R. Henry, I. Goldberg, Formalizing Anonymous Blacklisting Systems, IEEE S&P, 2011.

[10] Apple, iCloud Private Relay Overview, December 2021.

[11] Cloudflare, privacypass-ts 0.9.0 (TypeScript) and pat-go (Go), GitHub.

[12] W3C, Inaccessibility of CAPTCHA, W3C Working Group Note.

[13] Redis documentation, SET command and Lua scripting (EVALSHA).

Thank You for Your Attention!

TokenForge • A3-PS024-TC119 • AIORI Hackathon 3